The Agent Sleep Cycle

A memory architecture for coding agents: sessions work, offline phases consolidate — and every write is verified against source, scoped, and reversible.

wake · interactive | sleep · offline | memory · tiers

WAKE — PARALLEL SESSIONS

Session A `project: one`
context window = fast, fragile memory

Session B `project: two`
reads memory via index · flags candidates

Session C `project: three`
unrelated topics never share a store

`session end → hook → queue`
wake does the work — never the bookkeeping

Charter + index loaded fresh at every session start — never compacted.

transcripts (raw) →

SLEEP — OFFLINE PHASES

01 Consolidate

per session · off-peak

1 extract → 2 verify vs raw source → 3 promote

- quality admission gate — admitting less beats more (71% vs 55%)
- never summarize a summary

One writer, one git commit per promotion — sessions hand off conclusions, never raw context.

02 Check

daily · scripts only — no model

- TTL expiry — memories age out unless re-validated
- pointer re-validation · founding-test fixtures
- sliding old-task scores ⇒ drift alarm

Read-only. Raises flags, rewrites nothing — caps the exposure window of any bad memory at ~1 day.

03 Reconcile

on trigger: N new memories · or contradiction flag

- dedupe · prune · scoped rewrites
- contradictions escalate to the human — never auto-merge

Never on a calendar. Every rewrite is a compression event; batch enough inflow to justify one.

04 Prestudy

idle · off-peak

- re-derive repo state → warm-start memory
- next session starts warm, not rediscovering

Rebuilt from source every cycle — regenerative by construction, so it structurally cannot drift.

verified facts serialized →

MEMORY — THREE TIERS

Tier 0 · context — dies with the session

promoted conclusions only ↓

Tier 1 · project memory

project A isolated store | project B isolated store

One fact per file · scope frontmatter · **MEMORY.md** index routes recall. Parallel sessions meet only as verified conclusions.

git — every write snapshotted · every change reversible

Tier 2 · charter — pinned

AGENTS.md + skill cores. Outside the sleep agent's write scope; re-loaded fresh each session. Unpinned rules get dropped by compaction 30–59% of the time — pinning holds 0%.

Recovery

Drift alarm ⇒ **restore last-known-good** from the git history. Failures are loud and early, never silent decline.

Succession

When re-grounding costs more than rebuilding: **regenerate from source.** A successor inherits the charter + tested memories — never the full store.

OPERATING RHYTHM

Check daily · rewrite on trigger

consolidate per session — check daily — reconcile on trigger — rebuild by generation

Every mechanism answers a documented failure mode

Nothing here is defensive decoration — each gate, tier, and schedule exists because a specific, measured degradation was demonstrated somewhere.

9 defenses · all evidence-linked

FAILURE MODE	DEFENSE	LIVES IN	EVIDENCE
Semantic drift "mildly spicy" consolidates into "loves very spicy"	Verify against the raw transcript and repo — never summarize a summary	consolidate · pass 2	The AI Corner · arXiv 2605.16746
Procedural drift a workaround gets filed as the method and outlives the bug	Founding tests per skill; sliding old-task scores trip the drift alarm	check · fixtures	The AI Corner · arXiv 2605.09315
Governance decay compaction silently drops rules — 30–59% violations	Charter pinned outside the sleep agent's write scope — holds violations at 0%	charter tier	arXiv 2606.22528
Memory rot stale pointers and expired facts keep steering behavior	TTL expiry + daily re-validation; recency-weighted retention as the default	check · prune	Letta · The AI Corner
Cross-context bleed stored habits alter behavior in unrelated settings	Per-project isolation + scope frontmatter ("when this applies") on every file	memory tier	The AI Corner
Poisoning & laundering plant once, trigger later; bias survives compression looking clean	Conclusions-not-transcripts; cross-session content is untrusted; scrub before summarize	consolidate · gate 1	arXiv 2605.17830 · 2605.16746
Catastrophic forgetting edits to shared rules break previously working behavior	Additive, scoped skills — never rewrites of stable global instructions	skills + charter	SLEEP (2606.03979) · arXiv 2605.09315
Over-refinement repeated rewrites make memory generic and lossy	Rewrite only on inflow trigger — never on a calendar; batch first	reconcile	Letta · memory models
Plasticity–stability conflict global self-edits overwrite each other (r = −0.72)	Task-conditioned, scoped updates; frontmatter carries "when not to use"	all writes	Mila + DeepMind · 2610.25858

PER SESSION 01
Consolidate
extract → verify → promote, serialized

DAILY 02
Check
scripts only — expiry, pointers, fixtures, flags

ON TRIGGER 03
Reconcile
enough inflow, or a contradiction, buys one rewrite

PER GENERATION 04
Rebuild
regenerate from source; inherit charter + tested memories